HustleHub+

Part 1 - Secure Foundations

Secure Freelance Marketplace Platform

Backend API Implementation

INSY7314 - Portfolio of Evidence

Group members: ST10403582, ST10439005, ST10445479, ST10233318

Repository: https://github.com/ST10445479-ConnorBettridge/INSY7314_POE_HustleHubPlus

Contents

1. System Overview

HustleHub+ is a freelance marketplace that connects freelancers offering services with clients who want to book them. The platform handles payments between the two parties and tracks a freelancer's income, including an estimate of the tax owed on it.

The system is built on the MERN stack (MongoDB, Express, React, Node.js). Part 1 covers the secure foundations of the backend, so user data is held in memory for now and MongoDB is added in Part 2. Everything else in the security pipeline - HTTPS, hashing, tokens, validation, rate limiting - is fully implemented and tested.

Scope of Part 1

- User registration and login with hashed passwords
- Role-based accounts (Client, Freelancer, Admin)
- JWT-protected API routes
- All traffic served over HTTPS with TLS 1.2 as the minimum version
- Input validation and sanitisation on every request body
- Central error handling and structured logging
- Rate limiting and secure HTTP headers
- A native Android client that consumes the API over pinned HTTPS

2. Intended Users

The platform is built around three roles:

Role	Description	Permissions
Client	Browses services and books gigs	View gigs, book services, manage own bookings
Freelancer	Creates and manages gig listings	Create and manage gigs, view income and tax estimates
Admin	Runs the platform	Oversee users, manage transactions, system administration

Only Client and Freelancer can be chosen at registration. The Admin role is assigned internally and is rejected if a user tries to request it when signing up, which stops a new account from granting itself administrative access.

3. Architecture Diagram

The diagram below shows the layers of the system, the security controls applied at each one, and the boundary between the client and the server. It is also supplied as a separate vector file, architecture-diagram.svg, in the project root.

```
SYSTEM BOUNDARY
|
+-- CLIENT LAYER (Android app - Kotlin)
|     |
|     +-- HTTPS requests with JWT (Retrofit / OkHttp, pinned certificate)
|
+-- EXPRESS API SERVER (Node.js, HTTPS on port 3443)
|     |
|     +-- Middleware pipeline
|     |     +-- Helmet (secure HTTP headers)
|     |     +-- CORS
|     |     +-- Rate limiting (global and auth-specific)
|     |     +-- Body parser (10 KB limit)
|     |     +-- Input validation (express-validator)
|     |     +-- JWT authentication (protected routes only)
|     |     +-- Central error handler
|     |     +-- Logger (Winston)
|     |
|     +-- Routes
|           +-- Auth routes  (/register, /login, /profile)
|           +-- Gig routes          (Part 2)
|           +-- Transaction routes  (Part 2)
|
+-- DATA STORES
      +-- In-memory user store (Part 1)
      +-- MongoDB              (Part 2)
      +-- Winston log files
```

4. Architecture Explanation

The application separates concerns across four layers. Each one has a single responsibility, and a request passes through them in order.

4.1 Client Layer

The client is a native Android application written in Kotlin, in the android/ folder. It talks to the API over HTTPS only, at https://10.0.2.2:3443/ (the emulator's alias for the host machine). Every protected request carries the JWT in an Authorization: Bearer header. The app pins the backend certificate, so it will refuse to connect to any server that does not present that exact certificate, and it keeps the token in EncryptedSharedPreferences (AES-256-GCM) rather than in plain preferences.

4.2 API Layer (Express / Node.js)

The Express server is the core of the backend. Requests pass through the middleware pipeline in the order below before any business logic runs, so malformed or abusive requests are rejected as early as possible:

1. Helmet sets secure HTTP headers (CSP, HSTS, X-Frame-Options and others).
2. CORS controls which origins may call the API.
3. Rate limiting caps how many requests an IP address can make.
4. The body parser rejects payloads over 10 KB.
5. Route-level validation checks and sanitises every field.
6. JWT authentication runs on protected routes only.
7. The route handler executes the business logic.
8. The central error handler catches anything thrown and returns a safe response.

4.3 Security Layer

Security is applied at every layer rather than added at the end. The controls are:

- bcrypt password hashing (12 rounds), so stored credentials are never readable
- Stateless JWT authentication, verified on every protected request
- HTTPS with TLS 1.2 as the minimum version, so nothing travels in clear text
- Input validation with express-validator, which rejects malformed input before it is used
- Rate limiting, which slows brute-force and denial-of-service attempts
- Helmet, which sets security-focused HTTP headers and removes X-Powered-By
- Controlled error responses, which never return stack traces or file paths

4.4 Data Layer

User records are held in an in-memory store for Part 1 and will move to MongoDB in Part 2. The store exposes the same functions either way, so the routes will not need to change. Winston writes structured JSON logs to the logs/ folder, which is excluded from version control.

5. Backend Structure

The backend is organised as follows:

```
backend/
  server.js                  Entry point - creates the HTTPS server
  .env                       Environment variables (not committed)
  .env.example               Template for the above
  package.json
  certs/
    cert.pem                 Self-signed TLS certificate
    key.pem                  TLS private key
  logs/                      Winston output (generated at runtime)
  scripts/
    generate-cert.js         Self-signed certificate generator
  src/
    app.js                   Express app and middleware pipeline
    middleware/
      auth.js                JWT verification
      errorHandler.js        Central error handler and 404 handler
      validate.js            express-validator rule sets
    models/
      user.js                In-memory user store and bcrypt helpers
    routes/
      auth.js                Register, login and profile routes
    utils/
      logger.js              Winston configuration
  tests/
    auth.test.js             Endpoint tests
    security.test.js         Security regression tests
```

6. API Endpoints

Method	Endpoint	Auth	Description
GET	/	No	API welcome payload listing the available endpoints
GET	/api/health	No	Health check
POST	/api/auth/register	No	Create an account and return a JWT
POST	/api/auth/login	No	Authenticate and return a JWT
GET	/api/auth/profile	Yes	Return the authenticated user's profile

Any other path returns a 404 in the same JSON shape as every other error, so the client only ever has to parse one response format.

6.1 Request and Response Examples

Registration request - POST /api/auth/register

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "SecurePass1!",
  "role": "freelancer"
}
```

Registration response - 201 Created

```
{
  "status": "success",
  "message": "User registered successfully",
  "data": {
    "user": {
      "id": 1,
      "name": "John Doe",
      "email": "john@example.com",
      "role": "freelancer"
    },
    "token": "eyJhbGciOiJIUzI1NiIs..."
  }
}
```

The password hash is never included in a response. Only the four fields above are returned.

Failed login - 401 Unauthorized

```
{
  "status": "error",
  "statusCode": 401,
  "message": "Invalid email or password"
}
```

The same message is returned whether the email does not exist or the password is wrong. If the two cases returned different messages, an attacker could use the API to work out which email addresses are registered.

7. Security Decisions

7.1 Password Hashing

bcrypt with 12 salt rounds

Storing passwords in a form that cannot be reversed is the single most important control in an authentication system, because a database is the thing an attacker is most likely to end up with (OWASP, 2021). bcrypt was chosen over a general-purpose hash such as SHA-256 because:

- It is deliberately slow. At 12 rounds a single hash takes roughly a quarter of a second, so an attacker with a stolen database can only test a few passwords per second per core instead of millions (Provos & Mazieres, 1999).
- It salts automatically. Every password gets its own random salt, so two users with the same password end up with different hashes and precomputed rainbow tables are useless.
- The cost factor can be raised. As hardware gets faster the round count can be increased without changing the algorithm or invalidating existing hashes (Provos & Mazieres, 1999).
- It is well established. bcrypt has been public and peer-reviewed since 1999 and is still one of the algorithms OWASP recommends for password storage (OWASP, 2025a).

Passwords are hashed before they are stored, and bcrypt.compare() is used at login so the comparison takes the same amount of time whether or not the password is correct. When an email is not found, the code still runs a comparison against a dummy hash before returning the error, so a failed login takes the same time either way and cannot be used to discover which accounts exist. Plain-text passwords are never written to storage or to the logs.

7.2 Token-Based Authentication (JWT)

HS256 (HMAC with SHA-256)

A JSON Web Token is a signed, self-contained set of claims about the user, encoded so it can be passed safely in an HTTP header (Jones, et al., 2015). They were chosen for authentication because:

- They are stateless. No session table is needed, which keeps the API simple and lets it be run on more than one server without shared session storage.
- They are self-validating. The token carries the user id, email, role and name and is signed by the server, so a protected route can identify the caller without a database lookup.
- They expire. Tokens are issued with a one-hour lifetime (JWT_EXPIRES_IN), which limits how long a stolen token is useful.

A protected request is checked as follows:

9. The client sends Authorization: Bearer <token>.
10. The server verifies the signature against JWT_SECRET.
11. The server checks that the token has not expired.

12. The decoded payload is attached to req.user for the route handler.
13. Anything that fails these checks returns 401 with no detail about why.

The server refuses to start if JWT_SECRET is missing or shorter than 32 characters. Tokens signed with the wrong secret, tokens using the "none" algorithm, and expired tokens are all rejected, and there are tests covering each case. The "none" algorithm is worth calling out: it is permitted by the specification for unsecured tokens, so a library that honours the header without checking it will accept a token an attacker has simply rewritten (Jones, et al., 2015).

7.3 Input Validation

express-validator runs as middleware before each route handler (express-validator, 2025). The registration rules are:

Field	Rules
name	Required, trimmed, maximum 100 characters, HTML-escaped
email	Required, must be a valid address, normalised to lower case
password	Minimum 8 characters, and must contain an upper-case letter, a lower-case letter, a number and a special character
role	Must be exactly "client" or "freelancer"

Login validates the email format and checks that a password was supplied, but does not apply the strength rules - those only matter when a password is being set.

Validation matters because it is the boundary between untrusted input and the rest of the application, and failures here account for several of the entries in the OWASP Top 10, including injection and broken access control (OWASP, 2021). Escaping the name field means a payload such as <script>alert(1)</script> is stored and returned as harmless text rather than as markup. Restricting role to a fixed list stops a user from registering as an admin. Anything that fails is rejected with a 400 before the route handler runs, and unknown fields in the request body are ignored rather than copied onto the user record, which is the defence against mass assignment - where an attacker adds a property such as isAdmin to an otherwise ordinary request and the application binds it straight onto the model (OWASP, 2025c).

7.4 HTTPS Configuration

The server is created with https.createServer and will not accept plain HTTP at all. Details:

- Traffic is encrypted in transit, so credentials and tokens cannot be read off the network.
- TLS also protects integrity, so a request cannot be altered on the way to the server.
- minVersion is set to TLSv1.2, which rules out SSL 3.0 and TLS 1.0/1.1 - versions with known weaknesses that OWASP recommends disabling (OWASP, 2025b).

- The certificate is self-signed and generated locally with node-forge, which is appropriate for development. A production deployment would use a certificate from a trusted authority such as Let's Encrypt.
- The server listens on port 3443.

7.5 Additional Measures

The remaining controls follow the hardening steps Express recommends for production deployments (Express, 2025):

Control	Configuration	Attack it addresses
Helmet	Default header set, plus X-Powered-By disabled	Clickjacking, MIME sniffing, protocol downgrade
Rate limiting	100 requests per 15 minutes; 10 per 15 minutes on login and register	Credential brute force, denial of service
Body size limit	10 KB maximum, returns 413 above that	Large-payload denial of service
Error handling	Generic message for unexpected errors	Information disclosure through stack traces
Logging	Winston, server side only	Loss of an audit trail after an incident

Helmet is used with its default header set, which covers the headers most often flagged in a security scan (Helmet, 2025). Rate limiting is disabled while the test suite runs, otherwise the tests would trip the limit and fail for the wrong reason. It is active in every other environment.

8. Setup Instructions

Prerequisites

- Node.js v18 or later
- npm

Installation

```
cd backend
npm install
cp .env.example .env        # PowerShell: Copy-Item .env.example .env
node scripts/generate-cert.js
npm start                   # npm run dev for auto-reload
```

Open .env and replace JWT_SECRET with a random string of at least 32 characters before starting the server - it will exit with an error otherwise. One can be generated with:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

The API is then available at https://localhost:3443.

Browsers and Postman will warn about the self-signed certificate. That is expected in development; certificate verification can be turned off in Postman, or the certificate can be trusted locally.

Running the Tests

```
cd backend
npm test
```

9. Testing with Postman

The submission includes "HustleHub+ Postman Collection.json", a collection of 16 requests grouped into five folders. To use it:

14. Import the collection into Postman.
15. Set the baseUrl collection variable to https://localhost:3443.
16. Turn off SSL certificate verification (Settings > General), because the certificate is self-signed.
17. Run the folders in order. The login request saves the returned JWT to a collection variable, which the protected requests then reuse.

Folder	Requests	What it covers
Health	1	Server is up and responding
Registration	5	Valid sign-ups, duplicate email, bad input, admin role rejected
Login	5	Valid login, wrong password, unknown user, malformed JSON, oversized body
Protected Routes	4	Profile with a valid, missing, invalid and expired token
Security	1	Unknown route returns a controlled 404

Each request carries a test script that asserts the status code and the shape of the response, so the whole collection can be run at once with the Postman collection runner.

10. Automated Test Results

The backend is covered by Jest and Supertest across two suites: auth.test.js tests the endpoints, and security.test.js tests the specific attacks the security controls are meant to stop. All 28 tests pass.

10.1 Endpoint Tests (auth.test.js)

Group	Test	Result
Register	Registers a new freelancer	Pass
Register	Registers a new client	Pass
Register	Rejects a duplicate email	Pass
Register	Rejects a weak password	Pass
Register	Rejects an invalid email	Pass
Register	Rejects an invalid role	Pass
Register	Rejects an empty name	Pass
Login	Logs in with valid credentials	Pass
Login	Rejects the wrong password	Pass
Login	Rejects a non-existent email	Pass
Login	Rejects a missing password	Pass
Profile	Rejects access with no token	Pass
Profile	Rejects access with an invalid token	Pass
Profile	Returns the profile with a valid token	Pass
Health	Returns the health status	Pass
Root	Returns the API welcome payload	Pass
404	Returns 404 for an unknown route	Pass

10.2 Security Tests (security.test.js)

Group	Test	Result
Privilege escalation	Rejects self-registration as admin	Pass
Privilege escalation	Ignores unknown extra fields (mass assignment)	Pass
JWT attacks	Rejects a token signed with algorithm "none"	Pass
JWT attacks	Rejects a token signed with the wrong secret	Pass
JWT attacks	Rejects an expired token	Pass
JWT attacks	Rejects a token issued for the wrong context	Pass

Malformed input	Returns a controlled 400 for malformed JSON	Pass
Malformed input	Returns 413 for an oversized body	Pass
Injection	Stores and escapes SQL injection payloads safely	Pass
Response headers	Sends the expected security headers	Pass
Response headers	Leaks no internal detail on an unknown route	Pass

```
Test Suites: 2 passed, 2 total     Tests: 28 passed, 28 total
```

11. Android Client Application

A native Android client was built alongside the backend to show the Part 1 security requirements working from the client side as well. It lives in the android/ folder and was tested end to end against the running backend on an emulator.

11.1 Features

- Splash screen that routes to the dashboard or the login screen depending on whether a valid token is stored
- Registration with role selection (Client or Freelancer) and automatic login afterwards
- Login validated against the server, with the API's own 401 message shown on failure
- Dashboard showing the authenticated profile, the token, and a live security status panel
- Client-side validation that mirrors the server rules, so obvious mistakes are caught before a request is sent
- Token stored in EncryptedSharedPreferences (AES-256-GCM), not in plain preferences
- Certificate pinning against res/raw/server_cert.pem, so the app trusts only the HustleHub+ backend certificate

11.2 App Structure

```
android/
  build.gradle                      AGP 8.1.2, Kotlin 1.9.24
  app/
    build.gradle                    compileSdk 34, minSdk 26, targetSdk 34
    src/main/
      AndroidManifest.xml           INTERNET permission, network security config
      res/raw/server_cert.pem       Pinned backend certificate
      res/xml/network_security_config.xml
      res/layout/                   activity_login, activity_register,
activity_dashboard
      res/values/                   Theme and strings
      java/com/hustlehub/app/
        MainActivity.kt             Splash and authentication routing
        LoginActivity.kt            Login screen
        RegisterActivity.kt         Registration screen with role spinner
        DashboardActivity.kt        Profile, token, security status, logout
        HustleHubApplication.kt     Application context holder
        api/ApiClient.kt            Retrofit, OkHttp and the pinning TrustManager
        api/ApiService.kt           Endpoint definitions
        model/AuthModels.kt         Request and response data classes
        security/TokenManager.kt    Encrypted token storage
```

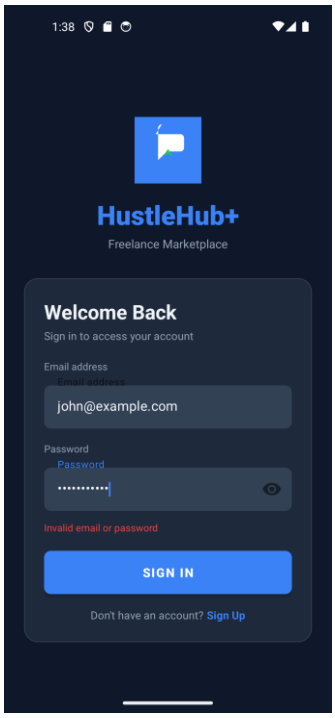
11.3 End-to-End Verification

The app was built with assembleDebug, installed on a Pixel 5 API 34 emulator and driven through the following checks against the live backend:

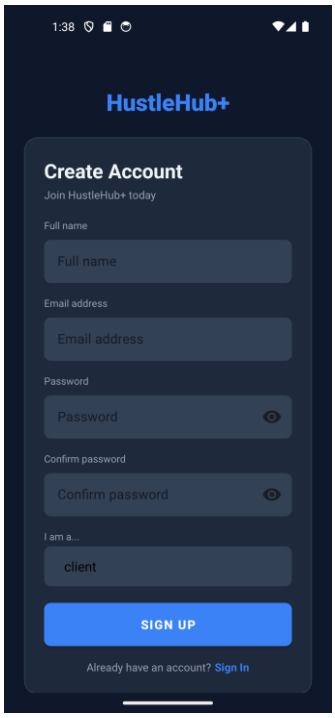
- Logging in with the wrong password shows the server's "Invalid email or password" message (401) rather than a generic client-side error.
- Registering a new freelancer returns 201 and moves straight to the dashboard.
- The dashboard shows the correct name, email, role and user id, and reports "Server: Connected".
- The token is displayed and stored encrypted; the security panel confirms HTTPS, bearer authentication, encrypted storage and bcrypt hashing.
- Logging out clears the token, and logging back in with correct credentials succeeds.
- Invalid emails and mismatched passwords are blocked before any network call is made.

11.4 Screenshots

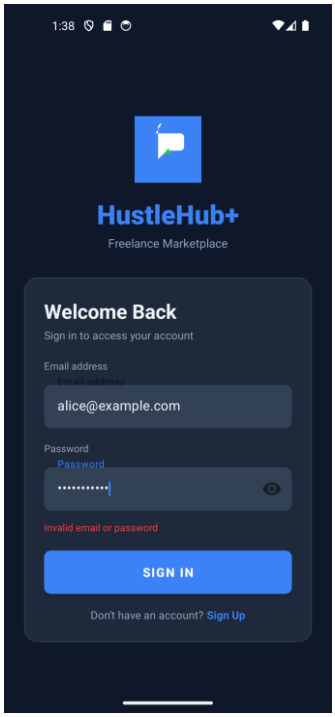
Captured from the emulator running against the live backend.



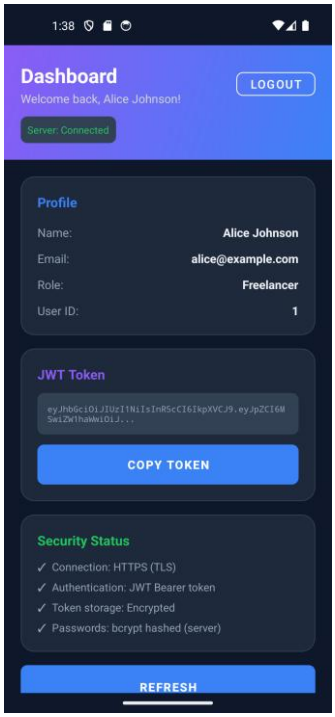
Login screen



Registration with role selection



Server-side 401 shown to the user



Dashboard with profile and security status

References

Express, 2025. *Production best practices: security*. [Online]
Available at: https://expressjs.com/en/advanced/best-practice-security.html
[Accessed 2026].

express-validator, 2025. *express-validator documentation*. [Online]
Available at: https://express-validator.github.io/docs/
[Accessed 2026].

Helmet, 2025. *Helmet: help secure Express apps with HTTP headers*. [Online]
Available at: https://helmetjs.github.io/
[Accessed 2026].

Jones, M., Bradley, J. & Sakimura, N., 2015. *RFC 7519: JSON Web Token (JWT)*. [Online]
Available at: https://www.rfc-editor.org/rfc/rfc7519
[Accessed 2026].

OWASP, 2021. *OWASP Top 10:2021*. [Online]
Available at: https://owasp.org/Top10/
[Accessed 2026].

OWASP, 2025a. *Password Storage Cheat Sheet*. [Online]
Available at:
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
[Accessed 2026].

OWASP, 2025b. *Transport Layer Security Cheat Sheet*. [Online]
Available at:
https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Security_Cheat_Sheet.html
[Accessed 2026].

OWASP, 2025c. *Mass Assignment Cheat Sheet*. [Online]
Available at:
https://cheatsheetseries.owasp.org/cheatsheets/Mass_Assignment_Cheat_Sheet.html
[Accessed 2026].

Provos, N. & Mazieres, D., 1999. *A Future-Adaptable Password Scheme*. [Online]
Available at: https://www.usenix.org/legacy/events/usenix99/provos.html
[Accessed 2026].

Appendix A: Postman Collection Export

The full collection, "HustleHub+ Postman Collection.json", is included with the submission. It contains 16 requests across five folders covering every endpoint together with the failure cases, and each request carries a test script that checks the status code and response body.